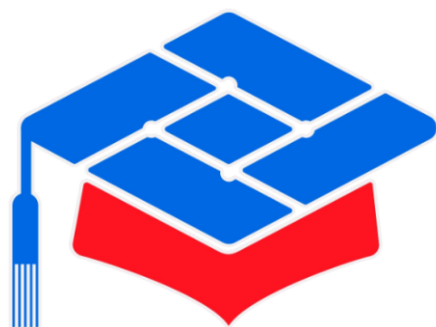




第十届

全国大学生集成电路创新创业大赛

报告类型：设计 SPEC 文档

参赛杯赛：叩持企业命题

作品名称：基于 RISC-V 的 CPU 模块设计

队伍编号：CICC100582

团队名称：白毛兽耳小萝莉

目录

第十届	- 1 -
全国大学生集成电路创新创业大赛	- 1 -
快速预览简介	- 5 -
系统整体架构设计	- 6 -
总体结构与工作设计	- 6 -
核心模块划分	- 6 -
架构特点	- 7 -
指令集体系规范	- 8 -
指令位域与基本含义	- 8 -
指令分类与字段判别规则	- 8 -
rs1,rs2 和 rd 的作用	- 10 -
与控制单元的关系	- 10 -
各功能模块详细规格	- 10 -
取指译码 (IF) 阶段	- 10 -
程序计数器 PC	- 10 -
下一条地址生成器 (NPC)	- 11 -
指令存储器 (IM)	- 12 -
指令寄存器 (IR)	- 12 -
译码与寄存器读取阶段 (ID) 功能模块	- 12 -
寄存器堆 (RF, Register File)	- 12 -
立即数拓展单元 (EXT)	- 13 -
全局控制单元 (Control Unit)	- 13 -
执行阶段 (EX) 功能模块	- 13 -
多路选择器 (MUX_2to1_A)	- 13 -
多路选择器 (MUX_3to1_B)	- 13 -
算术逻辑单元 (ALU)	- 13 -
访存与写回阶段 (MEM/WB) 功能模块	- 14 -
数据存储器 (DM, Data Memory)	- 14 -

级间流水寄存器（Flopr）	- 14 -
写回多路选择器（MUX_3to1_LMD）	- 14 -
信号与接口规范	- 15 -
顶层接口与全局约定	- 15 -
控制信号接口	- 15 -
译码字段接口	- 15 -
数据通路存储接口	- 15 -
预留接口与拓展性	- 16 -
时序与工作机制设计	- 17 -
复位与初始化	- 17 -
取指阶段	- 17 -
译码与执行阶段	- 17 -
访存阶段	- 17 -
写回阶段	- 18 -
分支与跳转	- 18 -
数据存储与寄存器规范	- 18 -
通用寄存器配置与读写规则	- 18 -
指令存储器（IM）地址映射与初始数据定义	- 20 -
数据存储器（DM）读写规则与地址对齐要求	- 20 -
标志位（零/进位/溢出）存储与更新规则	- 21 -
控制逻辑与状态机设计	- 22 -
梗概	- 22 -
控制器总体结构	- 22 -
状态机设计	- 22 -
关键控制逻辑	- 23 -
取指控制	- 23 -
运算与地址计算	- 23 -
写回控制	- 23 -
分支与跳转控制	- 23 -
设计特点	- 24 -

AI 工具使用声明	- 24 -
-----------------	--------

快速预览简介

本作品面向第十届集创赛初赛要求，围绕基于 RISC-V 的多周期 CPU 前端 RTL 设计展开，重点完成控制器状态机、时序与工作机制、核心数据通路以及功能仿真验证。当前实现已经形成“控制单元统一调度、数据通路分段锁存、分支跳转独立判断”的整体结构，能够覆盖赛题要求的主要指令行为。

总体方案采用多周期执行架构，将取指、译码、执行、访存和写回拆分为多个阶段，使不同类型指令按需占用不同时钟周期。与单周期实现相比，该方案显著缩短了单周期组合路径，降低了控制与运算的耦合度，也更适合在初赛阶段快速完成 RTL 设计与验证。

控制单元采用有限状态机进行调度，以 ReadIns、IRtypeCaculate、STOR_WB、LOAD_RMEM、LOAD_WB、JAL_WB、JALR_WB 和 B_JMP 等状态覆盖取指、普通运算、访存、跳转和分支处理。对 R 型与 I 型指令，设计允许在 IRtypeCaculate 状态下形成轻量重叠执行，即当前指令计算与下一条指令预取并行推进，从而提高指令吞吐效率。

在时序机制上，PC、IR、RF、DM 以及若干中间寄存器均以时钟上升沿作为主要边沿完成锁存。PC 在 PCWrite 有效时更新为 NPC，IR 在 IRWrite 有效时锁存 IM 输出，ALU_result、存储数据和返回地址则先进入 Flopr 再供后续阶段使用。这种“组合计算 + 边沿保存”的组织方式使各阶段职责清晰、时序边界明确。

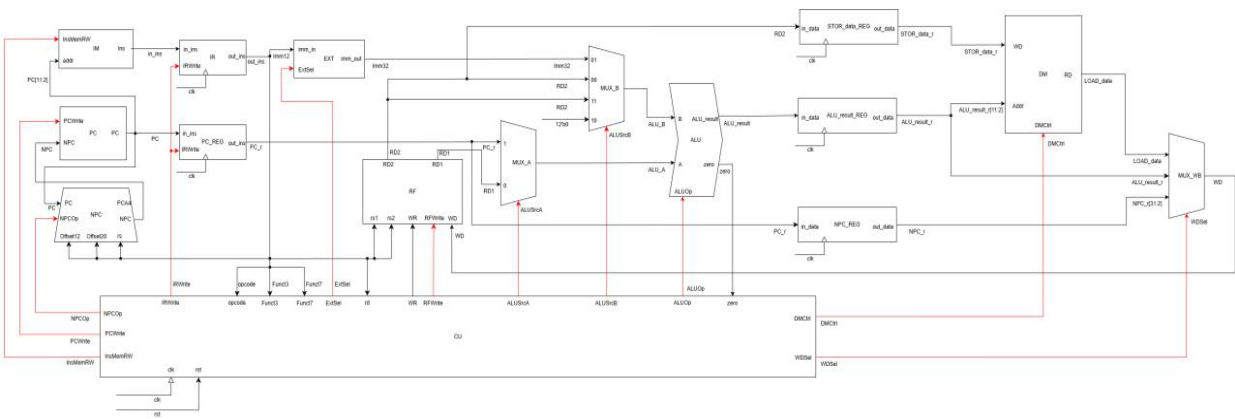
重要指标方面，本设计的数据通路宽度为 32 位，指令存储器与数据存储器均采用 $32\text{-bit} \times 1024$ 的存储结构，支持字对齐访问；控制逻辑覆盖 16 条 RISC-V 指令，兼顾寄存器寻址、立即数寻址、基址寻址、PC 相对寻址和伪直接寻址等常见模式。同时，寄存器堆 x0 恒为 0，保证指令语义与规范一致。

作品亮点主要体现在三点：其一，控制与数据通路分离明显，ControlUnit 统一产生 PCWrite、IRWrite、RFWrite、DMCtrl、NPCOp、ALUSrcA、ALUSrcB、ALUOp 和 WDSel 等关键控制信号；其二，分支与跳转采用独立处理路径，避免将比较逻辑与主运算链路过度耦合；其三，模块化 RTL 接口清晰，便于后续扩展和验证。功能仿真方面，当前设计已完成单指令功能验证与多指令组合场景验证，并可通过波形观察确认取指、译码、执行、访存和写回的控制时序符合预期。后续正文将围绕控制器状态机设计、时序与工作机制设计、RTL 实现细节以及仿真结果分

析展开说明，以完整展示本作品的设计思路与阶段性成果

系统整体架构设计

总体结构与工作设计



图表 1 总体结构框图

本设计采用多周期控制方式，整体按照取指、译码、执行、访存、写回五个逻辑阶段组织数据通路。控制单元统一调度各阶段的推进节拍，使不同类型指令按需占用不同数量的时钟周期，从而避免单周期结构中长组合路径带来的时序压力。在工作流程上，程序计数器首先给出当前取指地址，指令存储器返回指令原码，指令寄存器锁存当前指令后进入译码阶段。随后，寄存器堆提供源操作数，立即数扩展单元生成扩展立即数，算术逻辑单元完成运算、比较或地址计算；对于需要访存的指令，数据存储器在后续阶段完成读写；最终，写回选择器将运算结果、存储器读出值或链接地址送回寄存器堆。

对于普通算术类指令，设计允许在控制条件满足时形成局部重叠执行，即当前指令完成运算的同时，下一条指令可以进入取指准备阶段。对于访存、分支和跳转类指令，则通过专用状态逐步完成地址生成、比较判断、访存和结果写回，保证控制逻辑清晰、时序关系明确。

核心模块划分

从功能上看，整个 CPU 由取指单元、译码与寄存器读取单元、执行单元、访存单元、写回单元和控制单元组成。

取指单元由 PC、NPC、IM 和 IR 构成，负责提供当前指令地址、生成下一条地

址并完成指令锁存。译码与寄存器读取单元由 RF、EXT 和 ControlUnit 中的译码部分构成，负责识别指令字段、读取源操作数并生成控制信息。执行单元由多路选择器和 ALU 构成，负责在不同数据源之间切换，并完成算术、逻辑、移位、比较和地址计算。访存单元由 DM 和阶段寄存器构成，负责加载和存储操作。写回单元由写回多路选择器和寄存器堆组成，负责将最终结果写回目标寄存器。控制单元则贯穿整个系统，根据当前状态和指令类型统一发出控制信号。

各模块之间不直接耦合具体指令语义，而是通过统一的控制和数据接口完成协作。这样可以让控制逻辑集中在 ControlUnit 中实现，同时使数据通路保持模块化和可扩展

架构特点

本设计的架构特点主要体现在三点。

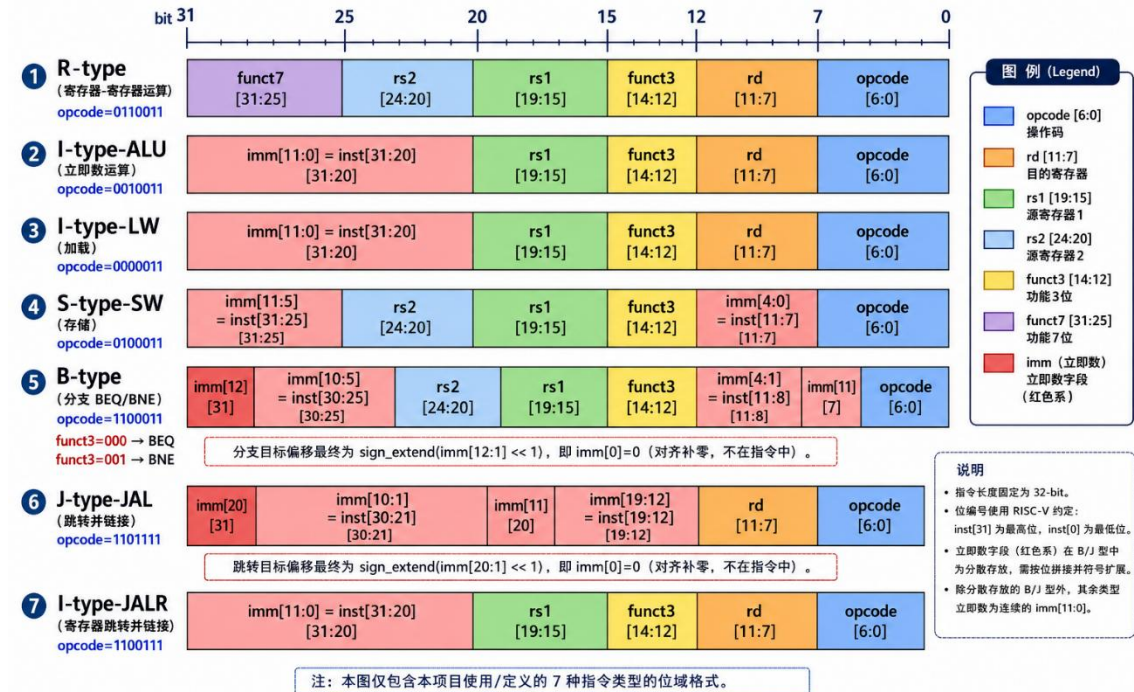
第一，控制与数据通路分离明显。控制单元只负责决定各阶段何时执行、如何选择数据源以及最终如何写回，数据通路只负责具体计算和传递，不在模块内分散实现复杂控制逻辑。

第二，阶段划分清晰。取指、译码、执行、访存、写回五个阶段职责明确，阶段之间通过寄存器保存中间结果，既方便调试，也有利于时序收敛。

第三，兼顾通用性与实现效率。对于常规算术类指令，系统允许局部重叠执行，提高整体吞吐率；对于访存、分支和跳转类指令，则采用专用状态拆分处理，避免长路径组合逻辑集中在单个周期内完成。

指令集体系规范

指令位域与基本含义



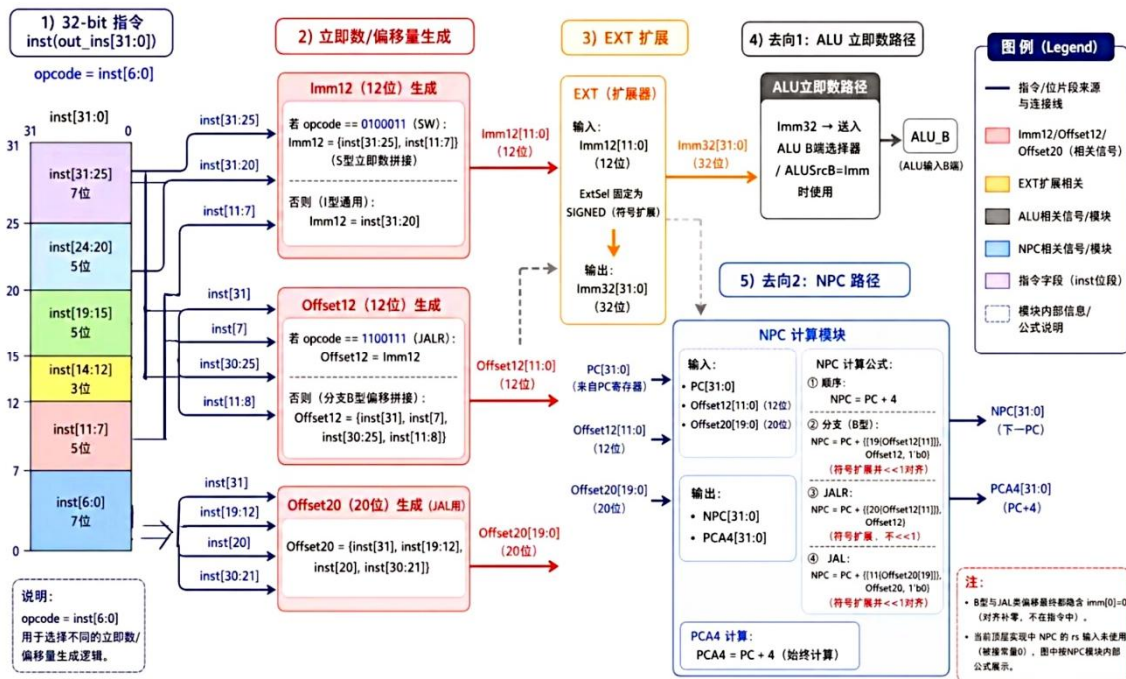
图表 2 指令位域格式图

如图 2 所示, 本设计采用 32-bit RISC-V 风格的指令位域划分: 从低位到高位依次为 opcode[6:0]、rd[11:7]、funct3[14:12]、rs1[19:15]、rs2[24:20] 和 funct7[31:25]。在 RTL 顶层中, 这些字段均由指令寄存器输出 out_ins[31:0] 直接切片拆分得到, 是控制单元完成指令识别与数据通路选择的基础: opcode 用于划分指令大类, funct3/funct7 用于进一步区分同类指令的具体功能, rs1/rs2 给出源操作数寄存器号, rd 给出写回目标寄存器号。基于该位域约定, 译码总体遵循“先看 opcode, 再看 funct3/funct7, 最后结合寄存器字段确定数据流”的判别顺序, 从而驱动后续的操作数选择、ALU 运算、访存控制与写回选择等关键控制信号生成。

指令分类与字段判别规则

当前 RTL 的指令识别遵循“先看 opcode, 再看 funct3/funct7, 最后结合寄存器字段确定数据流”的原则。对于 R 型指令, opcode=0110011, 控制单元在识别到该 opcode 后进一步结合 funct7 与 funct3 区分具体运算类型, 例如 0000000_000 对应 ADD、0100000_000 对应 SUB、0000000_111 对应 AND、0000000_110 对

应 OR、0000000_100 对应 XOR、0000000_001 对应 SLL、0000000_101 对应 SRL、0100000_101 对应 SRA。对于 I 型算术指令，opcode=0010011，主要通过 funct3 区分不同功能（如 ADDI/SLLI/SRLI/ORI/XORI 等），一般使用 rs1 与立即数参与运算并将结果写回 rd。对于 LW 和 SW，opcode 可直接区分加载与存储：LW 的 opcode=0000011，SW 的 opcode=0100011，二者都以 rs1 提供基址并结合 12 位立即数生成有效地址，其中 SW 的立即数按 S 型格式由 out_ins[31:25] 与 out_ins[11:7] 拼接得到，LW 则直接使用 I 型立即数 out_ins[31:20]。对于 B 型分支指令，opcode=1100011，再由 funct3 细分 BEQ/BNE（000/001），其分支偏移量由 out_ins[31]、out_ins[7]、out_ins[30:25]、out_ins[11:8] 重组得到；同时分支目标地址满足 2 字节对齐要求，偏移最低位隐含为 0，在目标地址生成时以追加对齐位的方式体现。为便于统一说明不同类型立即数/偏移量的重组规则及其去向，本设计在顶层对立即数相关字段进行集中生成，并分别送往 EXT、ALU 与 NPC 等模块，整体关系如图 3 所示。



图表 3 立即数与偏移量的生成与去向

结合图 3 可见，立即数相关信号在实现中可归纳为三类：Imm12（I/S 型 12 位立即数口径）、Offset12（B 型分支偏移口径，并与寄存器跳转类指令共用 12 位偏移通道）以及 Offset20（JAL 的 20 位偏移口径）。其中 Imm12 经 EXT 做符号扩展得到 Imm32，供 ALU 立即数运算与访存地址计算使用；Offset12/Offset20

送入 NPC 参与下一条 PC 的生成，并在 NPC 内部完成符号扩展与对齐位处理（B 型与 JAL 的偏移最低位隐含为 0，在地址计算时追加 1'b0 体现对齐；JALR 类偏移不需要低位对齐）。对于 JAL 与 JALR，opcode 分别为 1101111 与 1100111：JAL 采用 UJ 型偏移量，由 out_ins[31]、out_ins[19:12]、out_ins[20]、out_ins[30:21] 重组得到以实现 PC 相对跳转；JALR 在当前 RTL 中使用 I 型立即数口径并复用 Offset12 通道，NPC 模块中预留了相应的跳转地址生成分支用于该类指令扩展说明。总体而言，图 3 给出了“立即数/偏移量如何由指令位段重组、以及它们分别进入 EXT/ALU/NPC 的路径关系”，为后续 EXT、MUX 与 NPC 等模块规格描述提供统一口径。

rs1,rs2 和 rd 的作用

rs1 和 rs2 决定操作数来源，rd 决定结果写回位置。对于 R 型指令，rs1 和 rs2 同时参与运算，结果写回 rd；对于 I 型算术指令，只需要 rs1 和立即数，结果写回 rd；对于 LW，rs1 用于给出地址基址，加载结果写回 rd；对于 SW，rs1 给出地址基址，rs2 提供待存储数据；对于 BEQ 和 BNE，rs1 与 rs2 用于比较；对于 JALR，rs1 给出跳转基址，rd 则保存返回地址。

与控制单元的关系

opcode、funct3 和 funct7 的组合不仅用于判别指令类别，也直接决定控制单元输出的 ALUOp、NPCOp、WDSel、RFWrite 和 DMCtrl 等信号。换句话说，字段判别的结果会进一步映射到具体的数据通路控制上，从而完成算术运算、访存、分支和跳转等不同功能。

各功能模块详细规格

取指译码（IF）阶段

程序计数器 PC

输入为 clk、rst、PCWrite 和 NPC，输出为当前指令地址 PC。PC 负责向系统提供当前取指地址，是整个指令流的起点。

```

21  `include "ctrl_signal_def.v"
22
23
24  module PC(
25      input clk,                //时钟信号
26      input rst,                //复位信号
27      input PCWrite,            //PC写使能信号
28      input [31:0] NPC,         //下条指令的地址
29      output reg [31:0] PC      //本条指令地址
30  );
31
32  always @(posedge clk or posedge rst) begin
33      // reset
34      if (rst) begin
35          PC <= 32'h0000_2000;    //复位后PC的值
36      end
37      else if (PCWrite) begin
38          PC <= NPC;             //修改指令地址
39      end
40  end
41
42  endmodule

```

图表 4 程序计数器与 PC 的更新逻辑

下一条地址生成器（NPC）

输入为 NPCOp、Offset12、Offset20、当前 PC 和寄存器跳转基址 rs，输出为下一条指令地址 NPC 以及顺序执行地址 PCA4。该模块负责在顺序执行、分支跳转、JAL 和 JALR 等不同地址生成方式之间切换。

```

rtl > V NPC.v > {} NPC
1  `include "ctrl_signal_def.v"
2  `include "instruction_def.v"
3
4  module NPC(
5      input  [1:0]  NPCOp,      // 控制信号
6      input  [11:0] Offset12,   // 比较指令的跳转偏移量
7      input  [19:0] Offset20,   // 跳转指令的跳转偏移量
8      input  [31:0] PC,         // 本条指令的地址
9      input  [31:0] rs,         // 跳转到子程序的地址
10     output reg [31:0] PCA4,    // PC+4
11     output reg [31:0] NPC      // 下一条指令的地址
12 );
13
14 always@(*) begin
15     case(NPCOp)
16         `NPC_PC:      NPC = PC + 4; // 顺序执行
17         // BEQ,BNE指令跳转，需要在立即数末位补零对齐，符号位扩展
18         `NPC_Offset12: NPC = PC + {{19{Offset12[11]}},Offset12, 1'b0};
19         // JALR指令跳转，立即数符号位扩展，不需要低位对齐
20         `NPC_rs:      NPC = PC + {{20{Offset12[11]}},Offset12};
21         // JAL指令跳转，立即数末位补零对齐，符号位扩展
22         `NPC_Offset20: NPC = PC + {{11{Offset20[19]}},Offset20,1'b0};
23     endcase
24     PCA4 = PC + 4; // 始终计算PC+4，用于链接寄存器保存返回地址
25 end
26
27 endmodule

```

图表 5 下一条地址生成器 NPC 的顺序执行、分支与跳转逻辑

指令存储器 (IM)

输入为 `addr` 和 `InsMemRW`，输出为 32 位指令 `Ins`。`addr` 使用 `PC[11:2]` 作为字地址索引，适配字对齐取指方式。

指令寄存器 (IR)

输入为 `in_ins`、`clk` 和 `IRWrite`，输出为 `out_ins`。IR 用于保存从 IM 取出的当前指令，供后续译码与控制使用。

译码与寄存器读取阶段 (ID) 功能模块

寄存器堆 (RF, Register File)

输入为 `RR1`、`RR2`、`WR`、`WD`、`RFWrite` 和 `clk`，输出为 `RD1`、`RD2`。`RR1` 和 `RR2` 用于读取两个源操作数，`WR` 和 `WD` 对应写回地址和写回数据，`RFWrite` 为写使能信号。寄存器堆提供两个读口和一个写口，满足双操作数指令的访问需求。

立即数拓展单元 (EXT)

输入为 `imm_in`，输出为 `imm_out`，扩展方式由 `ExtSel` 约定。该模块负责将指令中的立即数字段扩展为 32 位形式，以便参与 ALU 运算或地址生成。

全局控制单元 (Control Unit)

输入为 `clk`、`rst`、`zero`、`opcode`、`Funct7`、`Funct3` 和 `rd`，输出为 `PCWrite`、`IRWrite`、`InsMemRW`、`RFWrite`、`DMCtrl`、`ExtSel`、`ALUOp`、`NPCOp`、`ALUSrcA`、`ALUSrcB`、`RegSel`、`WDSel` 以及 `WR`。该模块根据指令类型和控制状态统一生成全局控制信号，并确定写回目标寄存器。

执行阶段 (EX) 功能模块

多路选择器 (MUX_2to1_A)

输入为 `RD1` 与 `PC`，输出为 `ALU_A`。该模块用于在寄存器操作数和当前 `PC` 之间选择 ALU 的第一操作数来源。

多路选择器 (MUX_3to1_B)

输入为 `RD2`、`Imm32` 和常数零，输出为 `ALU_B`。该模块用于在寄存器操作数、扩展立即数和常数零之间选择 ALU 的第二操作数来源。

算术逻辑单元 (ALU)

输入为 `A`、`B` 和 `ALUOp`，输出为 `ALU_result` 和 `zero`。ALU 负责完成算术、逻辑、移位和比较类运算，并向控制单元提供 `zero` 作为分支判断依据。

```

rtl > V ALU.v > ...
1  `include "ctrl_signal_def.v"
2  `include "instruction_def.v"
3  |
4  module ALU(A, B, ALUOp, zero, ALU_result);
5      input signed [31:0] A;
6      input signed [31:0] B;
7      input [3:0] ALUOp;
8      output zero;
9      output reg signed [31:0] ALU_result;
10
11     assign zero = (ALU_result == 32'b0);
12
13     always @(*) begin
14         case (ALUOp)
15             `ALUOp_ADD: ALU_result = A + B;
16             `ALUOp_SUB: ALU_result = A - B;
17             `ALUOp_AND: ALU_result = A & B;
18             `ALUOp_OR:  ALU_result = A | B;
19             `ALUOp_XOR: ALU_result = A ^ B;
20             `ALUOp_SRA: ALU_result = $signed(A) >>> B[4:0];
21             `ALUOp_SLL: ALU_result = A << B[4:0];
22             `ALUOp_SRL: ALU_result = A >> B[4:0];
23             `ALUOp_BR:  ALU_result = A - B; //分支指令用减法做比较, A==B则Zero=1
24             default:   ALU_result = 32'b0;
25         endcase
26     end
27
28 endmodule

```

图表 6 算术逻辑单元 ALU 运算与零标志生成逻辑

访存与写回阶段（MEM/WB）功能模块

数据存储单元（DM, Data Memory）

输入为 Addr、WD、clk 和 DMCtrl，输出为 RD。Addr 使用 32 位地址的字索引部分，WD 为待写入数据，DMCtrl 用于区分读写操作，DM 负责完成加载和存储指令的数据访问。

级间流水寄存器（Flop_r）

输入为 clk、rst 和 in_data，输出为 out_data。该类通用锁存单元用于保存阶段间的中间结果，顶层中用于承载 ALU_result、待存储数据以及与返回地址相关的数据。

写回多路选择器（MUX_{3to1_LMD}）

MUX_{3to1_LMD}: 输入为 ALU_result_r、LOAD_data 和 NPC_r[31:2]，输出为 WD。该模块用于在计算结果、存储器读出值和链接地址之间选择最终写回数据。

信号与接口规范

顶层接口与全局约定

本设计采用顶层统一连线、子模块分工实现的接口组织方式。顶层模块 `riscv` 仅保留全局时钟 `clk` 与复位 `rst` 两个外部输入，其余功能均通过内部控制信号、数据通路信号和存储接口信号完成互连。系统以 32 位数据通路为主体，寄存器地址相关信号统一采用 5 位宽度，指令字段按照 32 位 RISC-V 格式拆分，保证各模块之间的位宽和语义一致。

控制信号接口

控制信号由 `ControlUnit` 统一产生，主要包括 `PCWrite`、`IRWrite`、`InsMemRW`、`RFWrite`、`DMCtrl`、`ALUSrcA`、`ALUSrcB`、`NPCOp`、`WDSel`、`ALUOp`、`RegSel` 和 `ExtSel`。其中，`PCWrite`、`IRWrite`、`RFWrite`、`DMCtrl` 分别用于控制程序计数、指令锁存、寄存器写回和数据存储器访问；`ALUSrcA`、`ALUSrcB` 与 `ALUOp` 用于描述运算通路的选择和运算类型；`NPCOp` 与 `WDSel` 分别用于下一条地址生成和写回数据选择；`RegSel` 与 `ExtSel` 在设计层面保留为接口信号，便于后续扩展与统一管理。

译码字段接口

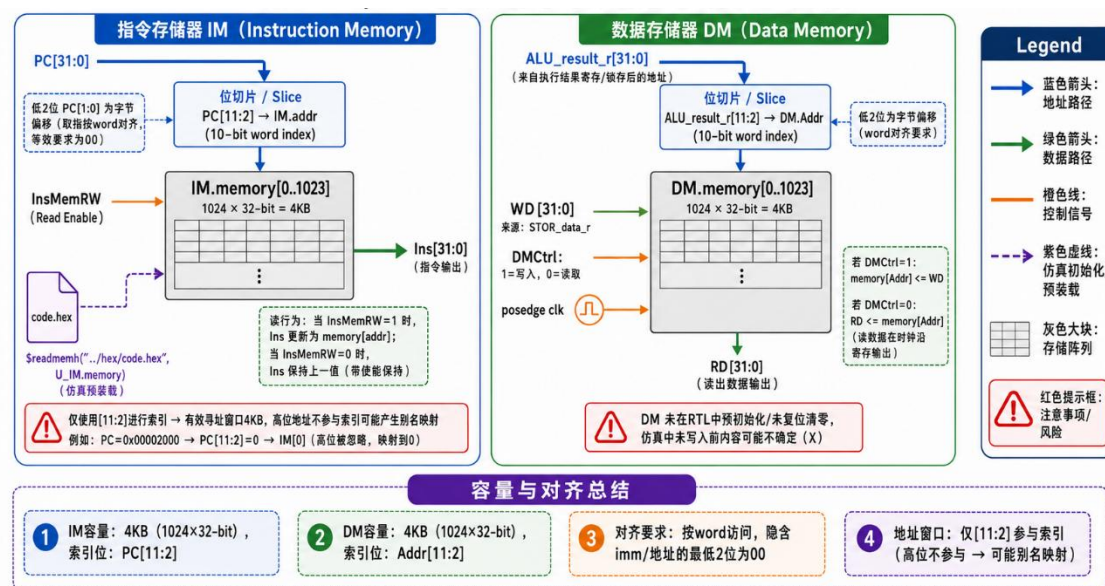
指令相关的核心字段包括 `opcode`、`Funct3`、`Funct7`、`rs1`、`rs2` 和 `rd`。这些信号都由 `out_ins` 拆分得到，并作为控制单元判断指令类型、选择运算路径、确定源寄存器和写回目标寄存器的重要依据。`opcode` 用于划分大类指令，`Funct3` 和 `Funct7` 用于进一步区分同类指令的具体功能，`rs1` 与 `rs2` 为源操作数地址，`rd` 为写回目标地址。

数据通路与存储接口

数据通路信号主要包括 `PC`、`NPC`、`PCA4`、`in_ins`、`out_ins`、`RD1`、`RD2`、`WD`、`ALU_A`、`ALU_B`、`ALU_result`、`ALU_result_r`、`STOR_data_r`、`NPC_r` 和 `LOAD_data`。`PC` 与 `NPC` 构成地址传递链路，`in_ins` 和 `out_ins` 构成指令传递链路，`RD1`、`RD2`、

ALU_A、ALU_B 和 ALU_result 构成运算数据链路，ALU_result_r、STOR_data_r、NPC_r 和 LOAD_data 则用于保存阶段之间的中间结果与写回相关数据。基于上述数据链路，取指与访存接口的关键差异集中体现在 IM/DM 的容量组织、地址索引位选择、对齐约束以及读写时序约定上，下面将结合图 4 对其进行说明。

指令存储器 IM 与数据存储器 DM 均采用 32-bit × 1024 的存储结构实现，按字（word）方式访问。IM 以 PC[11:2] 作为字地址索引，DM 以 ALU_result_r[11:2] 作为字地址索引，从而隐含最低 2 位为字节偏移并满足字对齐约束。IM 的读出由 InsMemRW 使能控制：当 InsMemRW=1 时根据地址更新输出指令；当 InsMemRW=0 时保持上一值。DM 采用时钟上升沿同步读写：DMCtrl=1 写入，DMCtrl=0 读出。上述容量、索引位、对齐与时序关系汇总如图 4 所示。



图表 7 IM 与 DM 地址映射与对齐规范

需要注意的是，由于 IM/DM 均仅使用地址的 [11:2] 参与索引，因此有效寻址窗口为 4KB，高位地址不参与译码时可能产生别名映射；同时 DM 在 RTL 中未做初始化/复位清零处理，仿真中未写入前的存储内容可能为不定值（X）。因此在编写测试用例时，应保证对 DM 的读操作之前已完成相应地址的写入或初始化，以避免不确定数据对结果分析造成干扰。

预留接口与拓展性

ExtSel 和 RegSel 在当前设计中保留为统一控制接口，其中 ExtSel 用于兼容不同类型立即数扩展方式，RegSel 用于兼容不同目标寄存器选择策略。当前实现已经

根据赛题要求完成了主要指令路径，因此这两个信号更多承担接口预留和后续扩展的作用，而不在本阶段作为重点展开对象。

时序与工作机制设计

复位与初始化

系统复位后，PC 被初始化为程序入口地址 0x00002000，控制状态回到初始取指流程。此时 PCWrite、IRWrite 等控制信号处于受控状态，系统从统一起点开始执行，为后续指令流建立稳定的初始环境。

取指阶段

取指阶段中，控制单元通过 PCWrite 和 IRWrite 协调 PC 更新与指令锁存，并由 NPCOp 选择下一条地址来源。顺序执行时，NPCOp 选择 PC+4；若遇到分支或跳转，则由 NPCOp 选择分支偏移、JAL 偏移或寄存器跳转地址。当前指令从指令存储器读出后，由 IR 在 IRWrite 有效时锁存，随后进入下一阶段译码。对于连续执行的普通算术类指令，PCWrite 和 IRWrite 还可以形成轻量重叠，使取指与前一条指令执行并行推进。

译码与执行阶段

译码阶段主要完成操作数准备和运算路径选择。ALUSrcA 决定 ALU 的 A 端输入来自寄存器操作数还是当前 PC，ALUSrcB 决定 B 端输入来自寄存器操作数、扩展立即数或常数零，ALUOp 则决定具体运算类型。对于 R 型和 I 型指令，ALU 在该阶段完成算术、逻辑或移位运算；对于 LW 和 SW，ALU 主要用于计算有效地址；对于分支类指令，ALU 执行比较运算并生成 zero 信号，为后续分支判断提供依据。

为了避免组合结果跨多个阶段直接传播，ALU 的输出会先经过阶段寄存器保存，再供后续访存或写回使用。这样既保证了数据稳定，也避免了过长的组合路径。

访存阶段

LW 和 SW 的处理都依赖前一阶段生成的有效地址。地址稳定后，DMCtrl 决定

数据存储器的访问方向：当 DMCtrl=1 时，数据存储器执行写操作；当 DMCtrl=0 时，数据存储器输出读数据。SW 使用 STOR_data_r 中保存的数据完成存储写入，LW 则在读出 LOAD_data 后等待后续写回。

这一阶段的核心是把“地址计算”和“数据访问”分开处理：地址由 ALU 先完成，真正的访存动作由 DMCtrl 控制，这样可以避免将整个加载/存储过程压缩到单个长组合链路中。

写回阶段

写回阶段由 WDSel 选择最终写回数据来源。普通 R 型和 I 型指令通常写回 ALU 结果；LW 写回来自数据存储器的 LOAD_data；JAL 和 JALR 则写回返回地址，即由 PC 相关路径得到的链接值。RFWrite 控制寄存器堆是否在时钟上升沿写入，而 WR 则在前序阶段提前锁存，保证写回时目标寄存器地址稳定，不会随译码变化而改变。

因此，写回阶段的关键不在于重新计算，而在于把前序阶段已经得到的结果通过 WDSel 和 RFWrite 正确送入寄存器堆。

分支与跳转

分支和跳转类指令采用独立控制路径处理。JAL 和 JALR 不依赖比较结果，控制单元直接通过 NPCOp 选择目标地址，并在写回时把返回地址送回寄存器堆。BEQ 和 BNE 则先由 ALU 产生 zero 信号，再由控制单元在后续状态中结合 Funct3 和 zero 决定是否更新 PC。若条件成立，则 PCWrite 有效，PC 按 NPCOp 指定的分支偏移更新；若条件不成立，则保持顺序执行，由下一轮取指阶段完成 PC+4。这种处理方式把“比较结果”和“是否更新 PC”拆成两个控制动作，既降低了控制复杂度，也避免分支判断直接占用主运算路径。

数据存储与寄存器规范

通用寄存器配置与读写规则

本处理器采用 32×32-bit 通用寄存器堆 RF，内部以 reg [31:0] register[0:31] 实现 32 个通用寄存器。寄存器读操作为组合逻辑读：RD1/RD2 由读地址 RR1/RR2 直

接索引得到，可在同一周期内提供给后级运算单元使用；写操作为时钟同步写：在 `posedge clk` 时若 `RFWrite=1` 且 `WR!=0`，将写数据 `WD` 写入 `register[WR]`。为保证 `x0` 恒为 0，模块中对 `register[0]` 进行持续置零（`always @(clk) register[0]=0`），且写入逻辑显式屏蔽 `WR==0` 的情况，从而实现对 `x0` 的硬约束。此外，为缓解“同周期写后读（RAW）”导致的读数滞后，读端加入旁路：当 `WR==RR1`（或 `WR==RR2`）且 `WR!=0` 时，`RD1`（或 `RD2`）直接输出当前待写入的 `WD`，保证读到的是最新值。

```

rtl > V RF.v > ...
1
2  `include "ctrl_signal_def.v"
3  `define DEBUG 1
4
5
6  module RF(
7      input  [4:0]  RR1,      // 读取寄存器1地址
8      input  [4:0]  RR2,      // 读取寄存器2地址
9      input  [4:0]  WR,      // 写入寄存器地址
10     input  [31:0] WD,      // 写入数据
11     input          RFWrite,  // 寄存器写使能信号
12     input          clk,      // 时钟信号
13     output [31:0] RD1,      // 读取寄存器1数据
14     output [31:0] RD2      // 读取寄存器2数据
15 );
16
17 reg [31:0] register [0:31]; // 32个32位寄存器
18
19 // 硬编码x0寄存器为0，忽略对x0的写入
20 always @(clk) begin
21     register[0] = 32'h0;
22 end
23
24 always @(posedge clk) begin
25     // 当写使能有效且写入地址非0时，执行写入操作
26     if ((WR != 0) && (RFWrite == 1)) begin
27         register[WR] <= WD;
28     end
29 end
30
31
32 // 组合逻辑读取寄存器数据
33 assign RD1 = (WR==RR1&&WR!=0)?WD:register[RR1];
34 assign RD2 = (WR==RR2&&WR!=0)?WD:register[RR2];
35
36
37 `ifdef DEBUG
38     always @(*) begin
39         // DEBUG模式下打印寄存器组状态
40         $display("%t", $time);
41         $display("R[00-07]=%8X %8X %8X %8X %8X %8X %8X %8X",
42             register[0], register[1], register[2], register[3],
43             register[4], register[5], register[6], register[7]);
44         $display("R[08-15]=%8X %8X %8X %8X %8X %8X %8X %8X",
45             register[8], register[9], register[10], register[11],
46             register[12], register[13], register[14], register[15]);
47         $display("R[16-23]=%8X %8X %8X %8X %8X %8X %8X %8X",
48             register[16], register[17], register[18], register[19],
49             register[20], register[21], register[22], register[23]);
50         $display("R[24-31]=%8X %8X %8X %8X %8X %8X %8X %8X",
51             register[24], register[25], register[26], register[27],
52             register[28], register[29], register[30], register[31]);
53     end
54 `endif
55
56 endmodule

```

图表 8 通用寄存器堆 RF 的读写、旁路与 x0 约束逻辑

指令存储器（IM）地址映射与初始数据定义

指令存储器 IM 采用 reg [31:0] memory[0:1023] 的 1024×32-bit 存储阵列（容量 4KB），输入地址端口为 addr[11:2]，即以“字”为单位寻址，天然满足 4 字节对齐的取指要求。顶层将 PC[11:2] 直接接入 IM 地址口，因此 PC 的低 2 位不参与索引；配合 PC 复位初值 0x0000_2000，可得到 0x2000 对应 PC[11:2]=0，即程序入口映射为 memory[0]，随后顺序执行对应 memory[1]、memory[2]…。IM 的读出逻辑为“地址敏感且受使能门控”：在 always @(addr or InsMemRW) 中仅当 InsMemRW=1 时执行 Ins <= memory[addr]，当 InsMemRW=0 时输出 Ins 保持不变（带使能保持行为）。本工程 IM 模块内部未固化初始内容，仿真时由测试平台通过 \$readmemh("../hex/code.hex", U_RISCV.U_IM.memory) 将 code.hex 预装载到 IM 存储阵列，从而定义指令初始数据；

```
21 `include "ctrl_signal_def.v"
22
23
24 module IM(
25     input      InsMemRW,    //指令存储单元信号
26     input [11:2] addr,      //指令存储器地址
27     output reg [31:0] Ins    //取得的指令
28 );
29
30     reg [31:0] memory[0:1023];
31
32     always @(addr or InsMemRW) begin
33         if (InsMemRW) begin
34             Ins <= memory[addr];    //根据地址取指令
35         end
36     end
37
38 endmodule
```

图表 9 指令存储器 IM 的字对齐取指逻辑

数据存储器（DM）读写规则与地址对齐要求

数据存储器 DM 同样采用 reg [31:0] memory[0:1023] 的 1024×32-bit 阵列（容量 4KB），地址端口为 Addr[11:2]，表示以“字”为单位索引，因此该存储系统仅支持 4 字节对齐的 word 级访问；外部计算得到的 32-bit 字节地址在进入 DM 前实际只取其中的 [11:2] 作为下标，低 2 位被丢弃，高于 bit11 的位在当前实现中不参与索引，等效为在 4KB 地址窗口内访问（超出窗口的高位变化会产生别名映射）。

DM 采用严格同步时序读写：在 `always @(posedge clk)` 中，当 `DMCtrl=1` 时执行写 `memory[Addr] <= WD`；当 `DMCtrl=0` 时执行读 `RD <= memory[Addr]`，因此读数据 `RD` 为寄存输出（同步读）。

```
rtl > V DM.v > {} DM
1  `include "ctrl_signal_def.v"
2  `define DEBUG 1
3
4
5  module DM(
6      input  [11:2] Addr,    //读写对应的地址
7      input  [31:0] WD,      //写入的数据
8      input      clk,        //时钟信号
9      input      DMCtrl,     //高电平写入，低电平读取
10     output reg [31:0] RD    //读出的数据
11 );
12
13     reg [31:0] memory[0:1023];
14
15     always @(posedge clk) begin //信号上升沿
16         if (DMCtrl) begin
17             memory[Addr] <= WD; //写入数据
18         end
19         else begin
20             RD <= memory[Addr]; //读出数据
21         end
22     end
23
24     `ifdef DEBUG
25         always @(*) begin
26             if(DMCtrl)
27                 $display("%t: mem[%h]=%h", $time, Addr, memory[Addr]);
28         end
29     `endif
30 endmodule
```

图表 10 数据存储器 DM 的同步读写控制逻辑

标志位（零/进位/溢出）存储与更新规则

本设计采用“最小标志位”策略：仅实现零标志 `zero`，用于分支比较等控制决策；未实现进位(Carry)与溢出(Overflow)标志，也没有独立的状态寄存器(如 `PSW/SR`)对标志位进行跨周期保存。零标志由 `ALU` 组合逻辑直接给出：`zero = (ALU_result == 0)`，即当 `ALU` 运算结果为全零时 `zero=1`。在分支比较场景中，`ALU` 通过执行 `A-B`（分支比较通道）产生结果，当 `A==B` 时 `ALU_result=0`，从而 `zero=1`，供控制单元选择是否进行 `PC` 相对跳转。由于 `carry/overflow` 不存在，报告中应明确说明：本 `CPU` 仅支持 `zero` 参与分支/比较控制，其它标志位不作为体系结

构状态的一部分。

控制逻辑与状态机设计

梗概

本多周期 CPU 设计由 ControlUnit 负责根据当前状态、opcode、Funct3、Funct7 以及分支比较结果 zero 生成控制信号。与单周期方案相比，该设计将取指、译码/执行、访存和写回分离，并通过状态机在不同指令之间切换控制节拍；其中 R 型和 I 型指令在 IRtypeCaculate 状态下形成“执行当前指令、同时准备下一条指令”的重叠模式，极大提高了吞吐率。

控制器总体结构

控制器以状态寄存器 state 和 next_state 为核心，采用同步状态更新、组合逻辑译码的方式工作。状态转移主要由当前指令的 opcode 决定，而分支类指令还会结合 Funct3 与 ALU 输出的 zero 信号完成是否跳转的判断。控制器输出的核心信号包括 PCWrite、IRWrite、InsMemRW、RFWrite、DMCtrl、ALUSrcA、ALUSrcB、ALUOp、NPCOp、WDSel 以及 WR。

状态机设计

状态机共包含 8 个状态，覆盖取指、普通计算、存储、加载、跳转和分支处理。

状态	功能	关键控制行为
ReadIns	初始取指和顺序取指	PCWrite=1, IRWrite=1, InsMemRW=1, NPCOp 选择 PC+4
IRtypeCaculate	译码与执行，兼顾普通指令的重叠执行	R/I 型指令在此完成运算并可同时写回上一条指令；SW、LW、B 型、JAL、JALR 在此分流到专用状态
STOR_WB	SW 写存储器	DMCtrl=1，完成数据存储，无寄存器写回
LOAD_RMEM	LW 读数据存储器	访存地址已由 ALU 计算并锁存，等待读出数据
LOAD_WB	LW 写回	WDSel 选择存储器读出值，RFWrite=1

JAL_WB	JAL 写回	NPCOp 选择 JAL 偏移，WDSel 选择 PC+4，RFWrite=1
JALR_WB	JALR 写回	NPCOp 选择寄存器跳转地址，WDSel 选择 PC+4，RFWrite=1
B_JMP	BEQ/BNE 分支判断	依据 Funct3 和 zero_r 决定是否更新 PC

从状态流转看，普通 R/I 型指令在 IRtypeCaculate 中即可完成执行，并在同一周期对上一条指令完成写回，因此可连续运行；而 LW、SW、分支和跳转指令则因需要额外的访存或控制转移步骤，被拆分为多个状态逐步完成。

关键控制逻辑

取指控制

在 ReadIns 状态下，PCWrite、IRWrite 和 InsMemRW 同时有效，PC 递增并从指令存储器读取下一条指令。对于连续执行的 R/I 型指令，IRtypeCaculate 状态同样允许 PCWrite 和 IRWrite 有效，从而形成取指与执行重叠的节拍。

运算与地址计算

ALUSrcA 用于选择 ALU 的 A 端输入，来源可以是寄存器 rs1 或当前 PC；ALUSrcB 用于选择 B 端输入，可在 rs2、立即数和常数 0 之间切换。ALUOp 根据指令类型选择加、减、与、或、异或、逻辑左移、逻辑右移、算术右移或比较运算。对于 R 型指令，ALUOp 由 Funct7 与 Funct3 联合决定；对于 I 型算术指令，由 Funct3 决定；对于 LW 和 SW，则统一使用加法完成基址加偏移地址计算；对于 BEQ/BNE，则使用减法生成 zero 标志用于比较。

写回控制

WDSel 用于选择写回寄存器的数据来源：ALU 结果、数据存储器读出值或 PC+4。普通 R/I 型指令写回 ALU 结果；LW 在 LOAD_WB 状态写回存储器读出值；JAL 和 JALR 写回返回地址 PC+4。RFWrite 控制寄存器堆是否在时钟上升沿写入。为了适配多周期路径，WR 会在对应状态前预先锁存目标寄存器 rd，保证写回阶段目的寄存器地址稳定。

分支与跳转控制

JAL 和 JALR 通过 NPCOp 直接选择目标地址：JAL 采用 PC 相对偏移，JALR 采用寄存器基址加偏移。BEQ/BNE 则先由 ALU 完成比较，再将 Funct3 和 zero 采样到 fun3_r、zero_r 中，随后在 B_JMP 状态统一判断是否成立；若成立，则 PCWrite=1 且 NPCOp 选择分支偏移，否则保持顺序执行。这样做可以将比较结果与 PC 更新解耦，降低控制路径复杂度。

设计特点

控制与数据通路分离明显，顶层只负责连线，ControlUnit 统一产生控制信号。

R/I 型指令采用重叠式多周期执行，兼顾实现简单性和吞吐率。

LW、SW、分支、JAL、JALR 通过专用状态处理，避免单周期中长组合路径带来的时序压力。

分支判断引入状态寄存的 Funct3 和 zero，确保跳转决策在正确的时钟边沿完成。

AI 工具使用声明

AI 工具名称、版本：chatgpt 5.4mini

使用场景与用途:使用其协助了文档撰写，包括提供一些写作思路和部分图片的润色。（使用占比 10%以内）